Tarea 2: Torneo de Komikan Concurrente

Informe de Diseño y Análisis de Sincronización

Daniel Carmona – Ricardo Vargas

1. Diseño del Esquema de Sincronización

Para implementar el torneo utilizamos hilos POSIX. Cada partida tiene un hilo para el Puma, doce hilos para los Perros y un hilo árbitro. Los hilos de las fichas intentan moverse de manera independiente, sin turnos fijos, pero comparten el mismo tablero. Por esta razón, utilizamos un *mutex* por partida para protegerlo.

Antes de realizar un movimiento, cada hilo obtiene el mutex y comprueba que el movimiento sea válido. Mientras mantiene el bloqueo actualiza las posiciones, las capturas y el estado de la partida. Así evitamos que dos hilos modifiquen el tablero al mismo tiempo o que un Perro capturado vuelva a moverse. Los retardos aleatorios se realizan fuera de la sección crítica para no bloquear innecesariamente a las otras fichas.

El árbitro se coordina con las fichas mediante la variable de condición `avisoArbitro` y el estado `capturaPendiente`. Cuando el Puma captura un Perro, el árbitro recibe el aviso, registra la captura y muestra el tablero actualizado. De esta forma se evita que los movimientos continúen mientras se está procesando ese evento.

Por otra parte, el administrador del torneo tiene su propio mutex para proteger la ocupación de las mesas y los contadores. Si las M mesas están ocupadas, las parejas esperan con `pthread_cond_wait` hasta que alguna quede disponible. También se registran las partidas activas para poder detenerlas cuando llega Ctrl+C. La señal SIGINT se atiende mediante un hilo separado que utiliza `sigwait`.

2. Análisis de Riesgos de Concurrencia

El principal riesgo es una condición de carrera al consultar y modificar posiciones compartidas. Por ejemplo, para capturar un Perro hay que comprobar la ficha intermedia, verificar que el destino esté libre y actualizar varias posiciones. Si otro hilo pudiera mover una ficha durante ese proceso, el tablero quedaría inconsistente. El mutex por partida permite realizar estas operaciones de forma protegida.

Respecto al *deadlock*, un punto delicado es la interacción entre el mutex del administrador, los mutexes de las partidas y la comunicación con el árbitro. Para reducir este riesgo mantenemos las secciones críticas acotadas y utilizamos variables de condición: `pthread_cond_wait` libera temporalmente el mutex mientras el hilo espera. Además, el cierre del torneo notifica a los hilos y parejas que puedan estar esperando. Aun así, el orden de adquisición de bloqueos debe respetarse al modificar el código.

El *livelock* podría presentarse si las fichas siguen realizando movimientos válidos, pero ninguna partida alcanza una condición de victoria. La prioridad que damos a las capturas del Puma y los retardos aleatorios ayudan a que el juego avance, aunque no garantizan que termine en un tiempo determinado. No dejamos un límite artificial de movimientos, porque agregaría una condición de término ajena a las reglas.

Finalmente, puede existir *starvation* si algunos hilos acceden al mutex con mucha más frecuencia que otros. Para disminuir la contención, los retardos están fuera del bloqueo y las secciones críticas son cortas. Sin embargo, no implementamos una política de turnos ni una garantía estricta de equidad entre hilos.

3. Evaluación del Diseño y Rendimiento

Elegimos un solo mutex para proteger el tablero completo en lugar de utilizar varios por posición. Esto hace que los movimientos dentro de una partida se ejecuten de manera serializada, pero simplifica la implementación y evita problemas al realizar capturas. Preferimos asegurar la consistencia del estado antes que aumentar el paralelismo dentro de una misma partida. En cambio, las distintas mesas mantienen estados y mutexes independientes, por lo que pueden jugar simultáneamente.

En cuanto al *throughput*, el número máximo de partidas activas está limitado por M. Aumentar las mesas puede permitir procesar más parejas en el mismo período, siempre que haya jugadores suficientes y recursos disponibles. La latencia de una pareja depende de cuánto espera por una mesa y del tiempo que tarda su partida. La latencia de cada movimiento incluye los retardos aleatorios y la espera para obtener el mutex; en las capturas también influye el tiempo que utiliza el árbitro para registrar el evento. No medimos tiempos exactos, por lo que esta evaluación es principalmente cualitativa.

Respecto a la complejidad, el número de parejas e hilos que administra el torneo crece con la cantidad de jugadores, mientras que las comprobaciones de movimientos trabajan sobre un tablero de tamaño fijo (25 posiciones más 3 del trono). Esto mantiene sencilla la lógica de cada partida, aunque el uso de más mesas también implica ejecutar más hilos al mismo tiempo.

Durante las pruebas ejecutamos el torneo con distintas cantidades de jugadores y mesas. En particular, con 12 jugadores y 3 mesas se formaron 6 parejas, se alcanzaron 3 mesas ocupadas simultáneamente y se realizaron 2 partidas en cada mesa. Todas terminaron y el resumen mostró 0 mesas ocupadas al finalizar. También comprobamos el caso de 7 jugadores y una sola mesa, donde se formaron 3 parejas y un jugador quedó sin pareja. Además, probamos la interrupción mediante SIGINT y rutas de error al crear hilos.

En general, el diseño logra coordinar varias partidas y reutilizar las mesas sin superar el límite establecido. Su principal costo es que los movimientos de una misma partida no pueden modificar el tablero en paralelo, una decisión que consideramos razonable para mantener la sincronización simple y el juego consistente.